7 – ROS2 Advanced Concepts

Robotics and Computer Vision (BPC-PRP)

Course supervisor: Ing. Adam Ligocki, Ph.D.

Ing. Adam Ligocki, Ph.D.
Brno University of Technology
2026

Robotics and AI

Agenda

1. DDS & ROS2 Architecture
2. Node Discovery & Domains
3. Quality of Service (QoS)
4. Launch Files
5. Parameters
6. TF2 – Coordinate Transforms
7. URDF & Xacro
8. RViz2 & Markers
9. Simulators – Gazebo, Webots
10. Bag Files
11. Lifecycle Nodes & Composition

DDS & Architecture

ROS2 Architecture Layers

- Your code (rclcpp / rclpy)
- RCL – ROS Client Library (C core, language-independent)
- RMW – ROS Middleware Interface (abstraction over DDS)
- DDS Implementation – actual networking (Fast DDS, Cyclone DDS, ...)
  - Key: you can swap DDS vendor without changing your code
- export RMW_IMPLEMENTATION=rmw_cyclonedds_cpp
- ROS1 had a custom TCP/UDP protocol + rosmaster
- ROS2 delegates all transport to a proven industry standard

What is DDS?

- Data Distribution Service – OMG standard (same org as UML, CORBA)
- Designed for military, aerospace, finance – adopted by ROS2
- Key properties:
  - Publish-subscribe, data-centric (not message-centric)
  - Fully decentralized – no broker, no master
  - Built-in discovery – nodes find each other automatically
  - Quality of Service – fine-grained delivery guarantees
- Two specs: DDS (API) + DDSI-RTPS (wire protocol over UDP)
- Implementations: Fast DDS (default), Cyclone DDS, RTI Connext

DDS Concepts Mapped to ROS2

- DDS Domain ↔ ROS_DOMAIN_ID
- DDS DomainParticipant ↔ roughly a ROS2 Node (Context)
- DDS Topic ↔ ROS2 Topic
- DDS DataWriter ↔ ROS2 Publisher
- DDS DataReader ↔ ROS2 Subscription
- DDS QoS Policies ↔ ROS2 QoS Profiles
- DDS IDL types ↔ .msg / .srv / .action (via rosidl code generation)
  - You never call DDS API directly – RMW does it for you

Node Discovery

Discovery Mechanism

- No rosmaster! (biggest change from ROS1)
- DDS Simple Discovery Protocol – two phases:
  - 1. PDP – multicast 'I exist' announcements on the network
  - 2. EDP – exchange topic names, types, and QoS info
- Connections made only if QoS profiles are compatible
- Periodic heartbeats keep nodes aware of each other
- Graceful leave announced on shutdown
- ROS_DOMAIN_ID – isolates groups (0-101, maps to UDP ports)
  - Different domain = invisible to each other
- ROS_LOCALHOST_ONLY=1 – restrict discovery to loopback

Quality of Service

QoS – Why It Matters

- ROS1: one transport mode, no control over delivery guarantees
- ROS2: publisher & subscriber each declare QoS profile
- Incompatible profiles → no connection (silent failure!)
- Allows mixing real-time critical and best-effort data on same system
- Example: camera images = BEST_EFFORT, safety stop = RELIABLE
- Predefined profiles: sensor data, system default, services
- Debug: ros2 topic info /topic --verbose

QoS Policies Overview

- Reliability: RELIABLE (retransmit) vs BEST_EFFORT (may lose)
- History: KEEP_LAST(N) vs KEEP_ALL
- Durability: TRANSIENT_LOCAL (late joiners get data) vs VOLATILE
- Deadline: max expected period between messages
- Liveliness: AUTOMATIC vs MANUAL_BY_TOPIC
- Compatibility rules (publisher must offer >= subscriber requests):
  - RELIABLE pub + BEST_EFFORT sub = OK
  - BEST_EFFORT pub + RELIABLE sub = INCOMPATIBLE!
  - TRANSIENT_LOCAL pub + VOLATILE sub = OK
  - VOLATILE pub + TRANSIENT_LOCAL sub = INCOMPATIBLE!

QoS in Code

```
# Python
from rclpy.qos import QoSProfile, ReliabilityPolicy, DurabilityPolicy

qos = QoSProfile(
    reliability=ReliabilityPolicy.RELIABLE,
    durability=DurabilityPolicy.TRANSIENT_LOCAL,
    depth=10
)
self.pub = self.create_publisher(String, '/status', qos)

# C++
auto qos = rclcpp::QoS(10)
    .reliability(RMW_QOS_POLICY_RELIABILITY_RELIABLE)
    .durability(RMW_QOS_POLICY_DURABILITY_TRANSIENT_LOCAL);
auto pub = this->create_publisher<std_msgs::msg::String>("/status", qos);
```

- Both pub & sub must have compatible QoS
- Shortcuts: rclpy.qos.qos_profile_sensor_data, rclcpp::SensorDataQoS()

Launch Files

Launch Files Basics

- Start multiple nodes with one command
- Set parameters, remappings, namespaces per node
- Three formats: Python (.launch.py), XML (.launch.xml), YAML
- Python = most powerful (full language), XML = most readable
- Run: ros2 launch <package> <launch_file>
- Key features:
  - Arguments with defaults (CLI overridable)
  - Conditions – start node only if argument is true
  - Event handlers – restart on crash, log on exit
  - Include other launch files (composability)

Launch File Example (Python)

```python
from launch import LaunchDescription
from launch.actions import DeclareLaunchArgument
from launch.substitutions import LaunchConfiguration
from launch_ros.actions import Node

def generate_launch_description():
    return LaunchDescription([
        DeclareLaunchArgument('use_sim', default_value='false'),

        Node(
            package='my_robot',
            executable='controller_node',
            name='controller',
            parameters=[{'use_sim_time': LaunchConfiguration('use_sim')}],
            remappings=[('/cmd_vel', '/robot1/cmd_vel')],
            output='screen',
        ),
    ])
```

- ros2 launch my_robot start.launch.py use_sim:=true
- remappings redirect topics without code changes

Parameters

Parameters in ROS2

- Key-value pairs associated with a specific node
- Types: bool, int, double, string, arrays
- Declared in code with defaults, descriptions, and value ranges
- Set at startup (launch file, CLI) or changed at runtime
- Parameter change triggers callback → validate & react
- CLI tools:
  - ros2 param list / get / set / dump / load
- YAML files for batch configuration:
  - node_name: { ros__parameters: { speed: 1.5 } }
- ros2 param dump /node > params.yaml → edit → load back

TF2 – Coordinate Transforms

TF2 – Concepts

- Manages relationships between coordinate frames
- Tree structure – each frame has exactly one parent
- Transform = translation (x,y,z) + rotation (quaternion)
- Standard chain: map → odom → base_link → sensors
  - map → odom: published by SLAM/localization
  - odom → base_link: published by odometry
  - base_link → camera_link: static transform
- Published on /tf (dynamic) and /tf_static topics
- Lookup: 'where is camera_link in map frame?' – TF2 walks the tree
- Convention: X forward, Y left, Z up (REP 103/105)

TF2 – Broadcast & Lookup

```
# Broadcasting (publish a transform)
br = tf2_ros.TransformBroadcaster(self)
t = TransformStamped()
t.header.frame_id = 'odom'
t.child_frame_id = 'base_link'
t.transform.translation.x = x
t.transform.rotation = quaternion_from_yaw(theta)
br.sendTransform(t)

# Lookup (query a transform)
tf_buffer = tf2_ros.Buffer()
tf_listener = tf2_ros.TransformListener(tf_buffer, self)
try:
    t = tf_buffer.lookup_transform('map', 'camera_link', rclpy.time.Time())
except tf2_ros.LookupException:
    self.get_logger().warn('Transform not available')
```

- StaticTransformBroadcaster for fixed transforms (sensor mounts)
- Tools: view_frames (PDF tree), tf2_echo, tf2_monitor

https://docs.clearpathrobotics.com/docs/ros/tutorials/rviz/

/opt/ros/humble/share/clearpath_viz/rviz/robot.rviz* - RViz

File Panels Help

Move Camera

Displays

Global Options	
Fixed Frame	odom
Background Color	48; 48; 48
Frame Rate	30
Global Status: Ok	
Grid	☑
RobotModel	☑
TF	☑

Add Duplicate Remove Rename

Reset **Left-Click:** Rotate. **Middle-Click:** Move X/Y. **Right-Click/Mouse Wheel:** Zoom. **Shift:** More options.

31 fps

URDF & Xacro

URDF Overview

- Unified Robot Description Format – XML
- Defines: links (bodies), joints (connections), geometry
- Each link has: visual (display), collision (physics), inertial (mass)
- Joint types: fixed, revolute, continuous, prismatic
- Published on /robot_description (TRANSIENT_LOCAL QoS)
- robot_state_publisher reads URDF → broadcasts TF from /joint_states
- Limitations: no closed chains, no loops
- Xacro – macro language on top of URDF:
  - Variables, math, macros (reusable blocks), includes, conditionals
  - Best practice: always use Xacro, even for simple robots

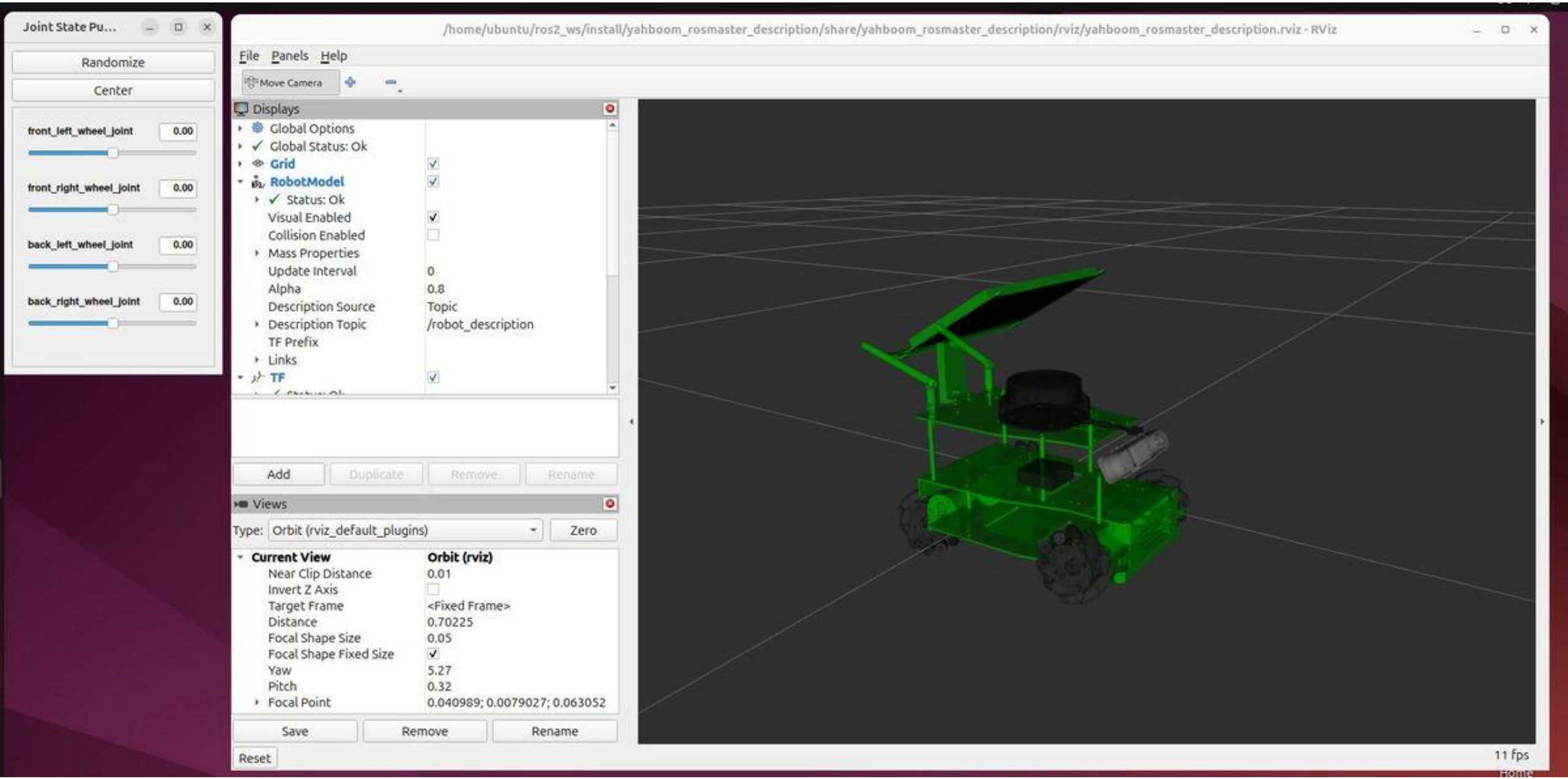
URDF Example

```
<robot name="simple_bot">
  <link name="base_link">
    <visual>
      <geometry><box size="0.5 0.3 0.1"/></geometry>
      <material name="blue"><color rgba="0 0 0.8 1"/></material>
    </visual>
    <collision><geometry><box size="0.5 0.3 0.1"/></geometry></collision>
    <inertial>
      <mass value="5.0"/>
      <inertia ixx="0.1" ixy="0" ixz="0" iyy="0.1" iyz="0" izz="0.1"/>
    </inertial>
  </link>

  <joint name="wheel_joint" type="continuous">
    <parent link="base_link"/>
    <child link="wheel_link"/>
    <origin xyz="0.2 0.15 -0.05"/>
    <axis xyz="0 1 0"/>
  </joint>
</robot>
```

- visual = what you see, collision = physics shape (can be simpler)
- inertial required for simulation (Gazebo will complain without it)

https://automaticaddison.com/create-launch-files-to-display-urdf-files-ros-2-jazzy/



RViz2 & Markers

RViz2 Overview

- 3D visualization tool – the 'eyes' of your ROS2 system
- Built-in display plugins: PointCloud2, LaserScan, Image, Map, Path, TF
- Configuration saved as .rviz files (load from launch: rviz2 -d config.rviz)
- Markers – custom visualization objects:
  - ARROW, CUBE, SPHERE, CYLINDER, LINE_STRIP, POINTS
  - TEXT_VIEW_FACING – labels always facing camera
  - MESH_RESOURCE – load 3D models (.dae, .stl)
- MarkerArray for publishing many markers efficiently
- Each marker needs: namespace + id (unique), frame_id, type, pose, scale, color
  - Common bug: color.a = 0 makes marker invisible!

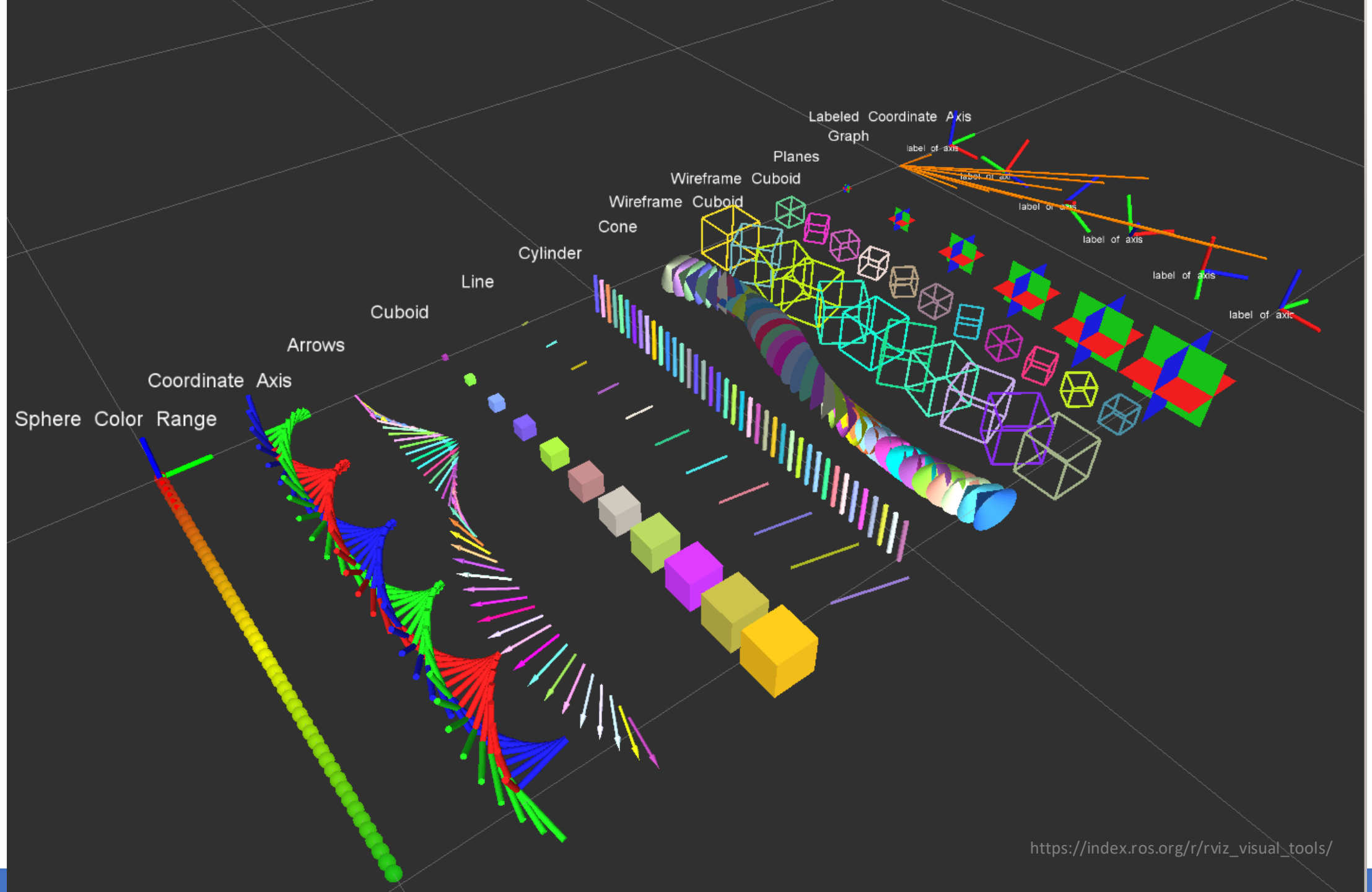
Publishing Markers

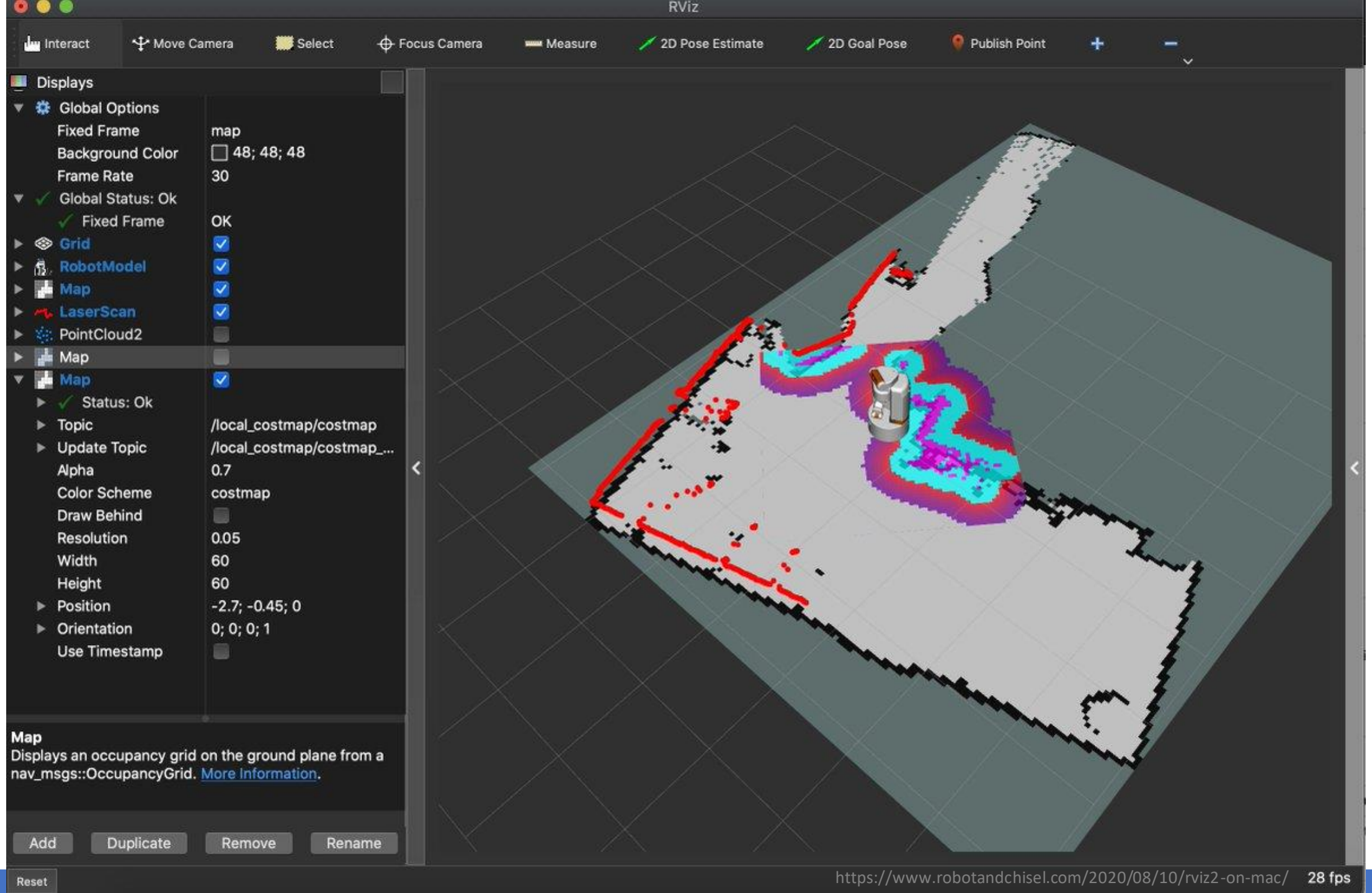
```
from visualization_msgs.msg import Marker

marker = Marker()
marker.header.frame_id = 'map'
marker.header.stamp = self.get_clock().now().to_msg()
marker.ns = 'obstacles'
marker.id = 0
marker.type = Marker.SPHERE
marker.action = Marker.ADD
marker.pose.position.x = 1.0
marker.pose.position.y = 2.0
marker.scale.x = 0.3   # diameter
marker.scale.y = 0.3
marker.scale.z = 0.3
marker.color.r = 1.0
marker.color.a = 0.8    # must be > 0!

self.marker_pub.publish(marker)
```

- Publish on topic of type visualization_msgs/Marker
- action: ADD=0, DELETE=2, DELETEALL=3
- lifetime=Duration(0) = permanent until deleted


Labeled Coordinate Axis
Graph
Planes
Wireframe Cuboid
Wireframe Cuboid
Cone
Cylinder
Line
Cuboid
Arrows
Coordinate Axis
Sphere Color Range
label of axis
label of axis
label of axis
label of axis
label of axis
label of axis
https://index.ros.org/r/rviz_visual_tools/



https://www.robotandchisel.com/2020/08/10/rviz2-on-mac/

Simulators

Gazebo vs Webots

Gazebo

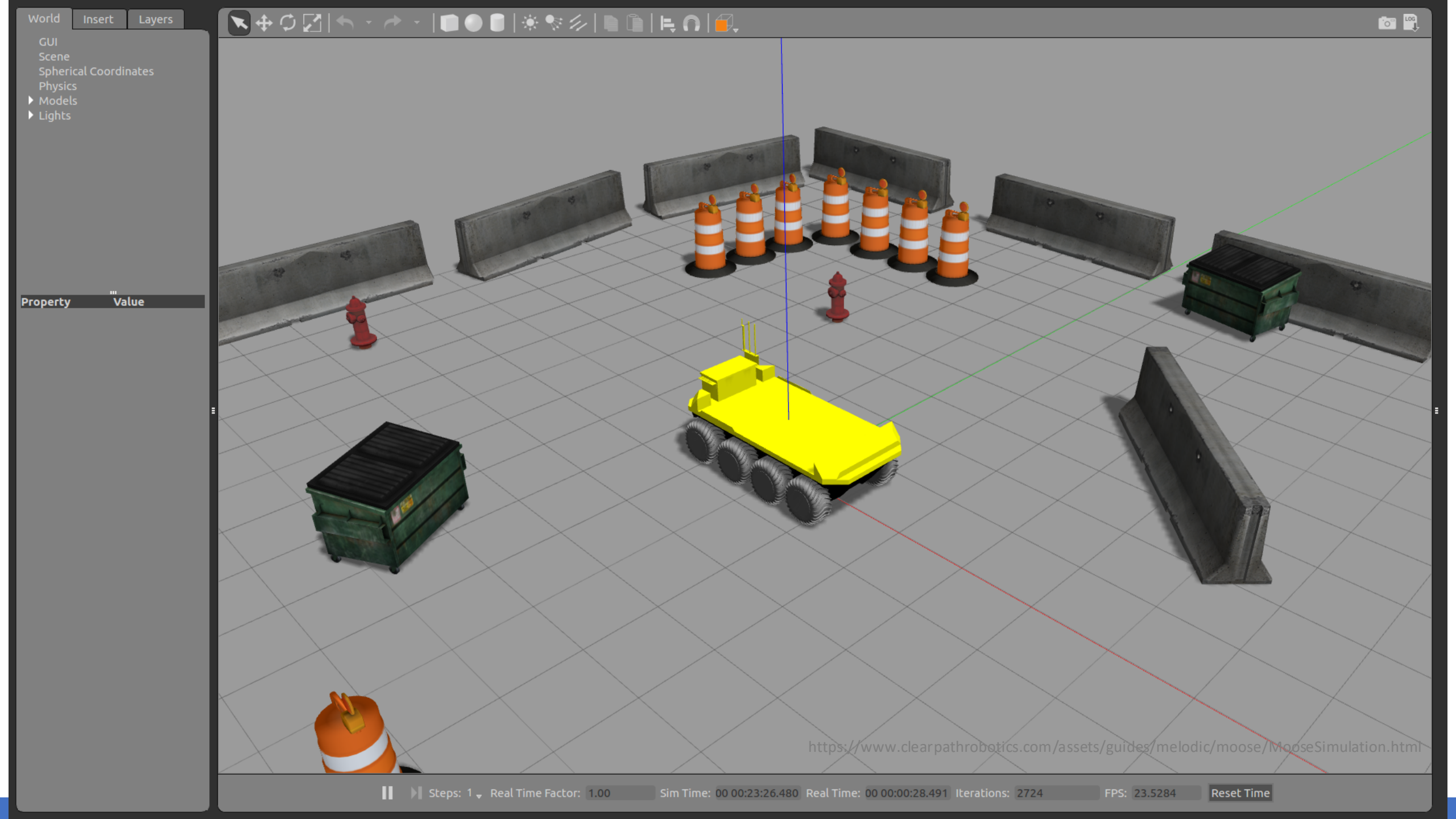
- Gazebo (Harmonic):
- Default ROS2 simulator
- SDF format (URDF auto-converted)
- Physics: DART, ODE, Bullet
- Plugin-based sensor/actuator system
- ros_gz_bridge for topic bridging
- Large model repository online
- use_sim_time for clock sync

Webots

- Webots:
- Open source (Cyberbotics)
- PROTO format + URDF import
- Built-in robot library
- Integrated IDE & controller editor
- webots_ros2 package
- Good tutorials & docs
- Cross-platform

Simulation Integration Tips

- Always set use_sim_time:=true for all nodes
- Gazebo publishes /clock – nodes use simulation time instead of wall clock
- Gazebo plugins publish standard ROS2 topics:
  - DiffDrivePlugin → /cmd_vel, /odom
  - CameraPlugin → /camera/image_raw
  - LidarPlugin → /scan
- spawn_entity – load URDF/SDF into running simulation
- Tip: keep your URDF valid for both RViz2 and Gazebo
- CI/CD: run automated tests in headless Gazebo


World
Insert
Layers
GUI
Scene
Spherical Coordinates
Physics
Models
Lights
Property
Value
https://www.clearpathrobotics.com/assets/guides/melodic/moose/MooseSimulation.html
Steps: 1
Real Time Factor: 1.00
Sim Time: 00 00:23:26.480
Real Time: 00 00:00:28.491
Iterations: 2724
FPS: 23.5284
Reset Time

Bag Files

Bag Files

- Record topic data for later replay and analysis
- Format: SQLite3 (.db3) + metadata.yaml (default), or MCAP
- Commands:
  - ros2 bag record /topic1 /topic2
  - ros2 bag record -a (record everything)
  - ros2 bag play <bag_dir> (replay)
  - ros2 bag info <bag_dir> (inspect: topics, duration, msg count)
- Playback options: --rate 2.0, --loop, --start-offset 10
- MCAP format: --storage mcap (indexed, seekable, Foxglove-friendly)
- Use cases: debugging, ML dataset creation, regression testing
- Foxglove Studio – great GUI for bag visualization

Lifecycle & Composition

Lifecycle Nodes

- Standard state machine for deterministic node behavior
- States: Unconfigured → Inactive → Active → Finalized
- Transitions triggered externally (CLI, launch, other nodes)
- Why? Without lifecycle: node publishes before subscribers are ready
- With lifecycle: configure all → activate all in correct order
- Callbacks: on_configure, on_activate, on_deactivate, on_cleanup
- create_lifecycle_publisher – only publishes when Active
- CLI: ros2 lifecycle set /node activate
- CLI: ros2 lifecycle get /node

Node Composition (Components)

- Run multiple nodes in ONE process (shared address space)
- Why? Avoid serialization overhead between nodes
- Intra-process communication: zero-copy message passing
- Huge win for image/pointcloud pipelines
- How it works:
  - Write node with NodeOptions constructor + register macro
  - Load into ComponentContainer at runtime or in launch file
- ComposableNodeContainer + ComposableNode in launch
- use_intra_process_comms: True → enables zero-copy
- CLI: ros2 component load /container my_pkg MyComponent

Adam Ligocki

Robotics and AI
Research Group

ligocki@vutbr.cz
https://github.com/Robotics-BUT
https://www.linkedin.com/in/adamligocki/
https://github.com/adamek727

Brno University of Technology
Faculty of Electrical Engineering and Communication
Department of Control and Instrumentation

Parallel Programming

Robotics and Computer Vision (BPC-PRP)

Course supervisor: Ing. Adam Ligocki, Ph.D.

Ing. Adam Ligocki, Ph.D.
Brno University of Technology
2026

Robotics and AI

Why Parallel Programming?

CPU clock speeds stopped growing around 2005. Instead, we got more cores.

Single-threaded

Like one cook in a kitchen.
Fast for simple meals, but
slow for a dinner party.

Multi-threaded

Multiple cooks working together.
Much faster — but they need to
coordinate or things go wrong.

4–128

cores in modern CPUs

1000s

of cores in a GPU

~5 GHz

clock speed wall since ~2005

Concurrency vs Parallelism

Concurrency

Dealing with many things at once

One barista, two orders
Makes latte, switches to espresso, switches back. Interleaving tasks.

Can happen on a single core. Tasks take turns — not truly simultaneous.

Parallelism

Doing many things at once

Two baristas, two orders
Each makes their own drink at the same time. True simultaneous.

Requires multiple cores. Tasks run at the exact same moment in time.

std::thread — Hello Parallel World

```
#include <iostream>
#include <thread>

void say_hello(int id) {
    std::cout << "Hello from thread " << id << "\n";
}

int main() {
    std::thread t1(say_hello, 1);  // launch thread
    std::thread t2(say_hello, 2);  // launch another

    t1.join();  // wait for t1 to finish
    t2.join();  // wait for t2 to finish
    return 0;
}
```

std::thread(fn, args...)

Creates a new OS thread and starts running fn.

.join()

Blocks until the thread finishes. Required!

Forgetting.join()?

Program crashes! std::terminate is called.

Output order is non-deterministic — run it 10 times, get different results!

Threads with Lambdas

```
#include <thread>
#include <vector>
#include <iostream>

int main() {
    std::vector<std::thread> threads;

    for (int i = 0; i < 4; i++) {
        threads.emplace_back([i]() {
            std::cout << "Thread " << i
                      << " is working...\n";
        });
    }

    for (auto& t : threads) {
        t.join();  // wait for all
    }

}
```

Lambda capture [i]

Captures i by value.
Each thread gets its own copy — safe!

Capture [&i] instead?

All threads share the same i — race condition!
It keeps changing.

Rule of thumb: capture by value unless you have a good reason not to.

Race Condition

Two threads modify the same data without coordination — result depends on who runs first.

Bank Account Example

```
Account balance: $100

Thread 1:                    Thread 2:
1. Read balance → $100
                              2. Read balance → $100
3. Write $100-$10 = $90
                              4. Write $100-$50 = $50

Final balance: $50
Expected:      $40   ← $10 lost!
```

```
int counter = 0;  // shared!

void increment() {
    for (int i = 0; i < 100000; i++)
        counter++;  // NOT atomic!
}

int main() {
    std::thread t1(increment);
    std::thread t2(increment);
    t1.join(); t2.join();
    // Expected: 200000
    // Actual: ??? (random!)
}
```

Deadlock

Two threads each hold a lock and wait for the other's lock. Neither can proceed — forever.

The Corridor Analogy

Two people in a narrow corridor:

Person A:
"I won't step aside until you step aside first."

Person B:
"I won't step aside until you step aside first."

→ Both stand there forever.
→ Program hangs. No crash, no error — just silence.

```
std::mutex m1, m2;

// Thread 1:
m1.lock();      // got m1
m2.lock();      // waiting for m2...
// do work
m2.unlock();
m1.unlock();

// Thread 2:
m2.lock();      // got m2
m1.lock();      // waiting for m1...
// do work       ← DEADLOCK!
m1.unlock();
m2.unlock();
```

std::mutex

```
#include <mutex>
int counter = 0;
std::mutex mtx;  // the "lock"

void safe_increment() {
    for (int i = 0; i < 100000; i++) {
        mtx.lock();
        counter++;   // only one thread at a time!
        mtx.unlock();
    }
}
```

Result

Now counter == 200000 every single time.

But what if...

An exception is thrown between lock() and unlock()? The mutex stays locked forever!

The fix: std::lock_guard (RAII)

```
void safe_increment() {
    for (int i = 0; i < 100000; i++) {
        std::lock_guard<std::mutex> guard(mtx); // auto-unlocks!
        counter++;
    } // guard destroyed here → mutex unlocked
}
```

lock_guard vs unique_lock

`std::lock_guard`

- Locks on creation
- Unlocks on destruction
- Cannot unlock early
- Cannot be moved
- Simpler, safer, preferred

Use this by default!

`std::unique_lock`

- Can lock/unlock anytime
- Can be moved between scopes
- Works with condition_variable
- Can try_lock (non-blocking)
- More flexible, slightly heavier

Use when you need fine control.

std::atomic — No Lock Needed

The CPU guarantees the operation happens in one indivisible step.

```
#include <atomic>

std::atomic<int> counter{0};  // atomic counter

void increment() {
    for (int i = 0; i < 100000; i++) {
        counter++;  // atomic increment!
        // same as: counter.fetch_add(1);
    }
}

int main() {
    std::thread t1(increment);
    std::thread t2(increment);
    t1.join(); t2.join();
}
```

Use atomic for simple counters and flags. Use mutex for everything else.

mutex

- Any data type
- Slower (OS overhead)
- Can deadlock
- Flexible

atomic

- Simple types (int, bool, ptr)
- Fast (hardware instruction)
- No deadlock possible
- Limited operations

std::async & std::future

"Go do this and give me the result when you're done." — No manual thread management.

```
#include <future>
#include <iostream>

int compute_answer(int x) {
    return x * x; // imagine heavy computation
}

int main() {
    auto fut = std::async(std::launch::async,
                          compute_answer, 42);
    std::cout << "Working on something else...\n";

    int result = fut.get(); // blocks if not ready
    std::cout << "Answer: " << result << "\n";
}
```

std::async

Runs a function in a separate thread (maybe).

std::future

A "ticket" for a result that isn't ready yet.

Why use this?

Simpler than raw threads. No join(), no shared state, just ask → get answer.

std::condition_variable

"Wait here until I signal you." — Producer/consumer pattern.

```
std::mutex mtx;
std::condition_variable cv;
bool data_ready = false;

// Producer thread
void producer() {
    {
        std::lock_guard<std::mutex> lk(mtx);
        data_ready = true;
    }
    cv.notify_one();  // wake up consumer
}

// Consumer thread
void consumer() {
    std::unique_lock<std::mutex> lk(mtx);
    cv.wait(lk, []{ return data_ready; });
    // data is ready, process it!
}
```

Why not just poll?

Busy-waiting wastes CPU:
while(!ready) {} // bad!
cv.wait() sleeps the thread.

Spurious wakeup?

The lambda predicate handles it — the thread rechecks the condition.

Golden Rules

Minimize shared state

The less data threads share, the fewer bugs.

Use atomic for simple types

Faster than mutex for counters & flags.

Prefer lock_guard

RAII locks — can't forget to unlock.

Prefer async over raw threads

Higher-level, less error-prone.

Lock in consistent order

Always lock A then B — prevents deadlocks.

Never access shared data unlocked

Even "just reading" can be undefined behavior.

C++ Parallel Toolbox — Cheat Sheet

Tool	What It Does	When to Use
std::thread	Runs a function in a new OS thread	When you need direct control
std::mutex	Exclusive lock for shared data	Protecting complex shared state
std::lock_guard	RAII wrapper — auto-unlocks mutex	Default choice for locking
std::unique_lock	Flexible lock — can unlock early	With condition_variable
std::atomic<T>	Lock-free thread-safe variable	Simple counters, flags, booleans
std::async	Launches async task, returns future	"Fire and forget" computations
std::future	Holds result of async computation	Getting results from async tasks
std::promise	Manually sets a future's result	Passing results between threads
std::condition_variable	Signal between threads (wait/notify)	Producer-consumer patterns
std::shared_mutex	Multiple readers OR one writer	Read-heavy shared data

Adam Ligocki

Robotics and AI
Research Group

ligocki@vutbr.cz
https://github.com/Robotics-BUT
https://www.linkedin.com/in/adamligocki/
https://github.com/adamek727

Brno University of Technology
Faculty of Electrical Engineering and Communication
Department of Control and Instrumentation